

# Reflexión Individual de la Solución Final:

Autor/a: Jaime García Algarra

Solución seleccionada para el merge a **master**: Propuesta de Mario

## 1. Justificación Técnica (Temas 1, 2 y 3)

Hemos elegido la propuesta de Mario como versión definitiva porque es la que mejor se adapta a los conceptos de la asignatura vistos hasta ahora, ciñéndose estrictamente a los apuntes de la asignatura:

- Tipos Sinónimos: En lugar de dejar tuplas sueltas, usa `type Posición = (Int, Int)` y `type Grid = [[Char]]`. Tal como vimos en el Tema 2, esto no crea tipos nuevos, pero hace que las firmas explícitas (`::`) sean muchísimo más legibles y documenten bien el código.
- Ajuste de Patrones y Guardas: En el cálculo de colisiones, huye del uso repetitivo de `fst` o `snd`. Hace *pattern matching* directamente sobre tuplas largas `(x, y, w, h)` y usa guardas (`|`), logrando un estilo puramente declarativo.
- Comprensión de Listas: Para buscar en la matriz (`posicionesMeta`), usa una comprensión combinada con `zip [0..]`. Es el ejemplo perfecto de cómo iterar y filtrar sin caer en los bucles imperativos tradicionales.
- QuickCheck y Clases Numéricas: Destaca por su uso de `import Test.QuickCheck`. Además, en la prueba de asociatividad, justifica perfectamente por qué falla en el tipo `Double` debido a los problemas de precisión de la clase `Fractional` vistos en el Tema 2.

## 2. Pros y Contras frente al resto del grupo

- A favor: Es la solución que más nos ha gustado respecto a lo visto en la asignatura; no introduce elementos avanzados no impartidos, tiene un *pattern matching* muy limpio y la mejor batería de pruebas QuickCheck.
- A mejorar: Al definir las firmas de algunas funciones numéricas auxiliares, Mario ha restringido el tipo directamente a enteros fijos (`Int`). Aunque funciona para la práctica, le resta generalidad al código. Habría sido un buen detalle aprovechar el polimorfismo mediante restricciones de clase como `(Num a) => o (Integral a) =>`, tal y como vimos en el Tema 2 para hacer las funciones más reutilizables.

## 3. Conclusión

Nos decantamos por hacer el merge con esta solución porque encapsula perfectamente lo que creemos que se espera de un alumno en este punto de PD. Demuestra que no hace falta usar estructuras complejas para hacer un buen código en Haskell; basta con aplicar bien los tipos sinónimos, exprimir el ajuste de patrones y apoyarse en la comprensión de listas. Es un código robusto, fácil de entender y sólidamente testado con QuickCheck, lo que evidencia que se han asimilado a fondo las bases de los tres primeros temas.